

**Lista enlazada simple frente a árbol n-ario para el seguimiento jerárquico de tareas de un
curso**

Kimberly Torres González y Gerald Vindas Ramírez

Tecnológico de Costa Rica

IC 2001: Estructuras de Datos

David Mora Fonseca

9 de octubre de 2026

Resumen

Un profesor universitario que administra tareas con una herramienta de seguimiento pasa de manejar tareas independientes, consultadas por identificador, a manejar proyectos descompuestos jerárquicamente en entregables y subtareas, donde además necesita conocer el esfuerzo total y pendiente de una tarea y de todos sus descendientes. Este estudio compara una lista enlazada simple, que registra la jerarquía solo mediante el identificador del padre, con un árbol n-ario enlazado en representación primer hijo–siguiente hermano. Se construyó un prototipo en C++11 que aplica ambas estructuras a una carga base y a una carga modificada de 100, 500 y 1000 tareas generadas con semilla fija, verifica cada respuesta contra un resultado esperado calculado de forma independiente y cuenta nodos visitados y comparaciones de identificadores. Las 10 032 comprobaciones fueron correctas. En la carga base ambas estructuras realizaron prácticamente el mismo trabajo de búsqueda. En la carga modificada con 1000 tareas, consultar el trabajo contenido en una tarea requirió en promedio 4309 visitas en la lista y 569 en el árbol, aunque construir el árbol costó 224 159 visitas frente a 203 867 en la lista. Se recomienda la lista para la carga base y el árbol para la carga modificada.

Palabras clave: estructuras de datos, lista enlazada, árbol n-ario, estructura de desglose del trabajo, carga de trabajo

Lista enlazada simple frente a árbol n-ario para el seguimiento jerárquico de tareas de un curso

Introducción

En un curso universitario, el profesor utiliza una herramienta de seguimiento para administrar las tareas que asigna a sus estudiantes. Al inicio del periodo lectivo el trabajo está formado por tareas independientes, como quices, prácticas y tareas cortas, cada una con un identificador, una estimación de horas de esfuerzo y un estado que indica si ya fue completada. En esa etapa las consultas son individuales: localizar una tarea por su identificador para revisar sus datos o actualizar su estado.

Conforme avanza el curso, el profesor asigna proyectos que se descomponen en entregables, como el informe y el video, y estos en tareas y subtareas, como redactar la metodología o grabar una sección. Con esta organización cambia la consulta: el profesor necesita saber cuántas horas de trabajo contiene un proyecto y cuántas quedan pendientes, lo que exige sumar el esfuerzo de todos sus descendientes, en todos los niveles. Una forma de organizar los datos que funciona al inicio del curso puede dejar de ser conveniente con esta nueva carga, y decidir requiere evidencia sobre la carga concreta y no solo sobre las estructuras aisladas.

Este trabajo corresponde al Tema 1, «Tareas y subtareas», del catálogo de la Investigación 01. Su pregunta central es la pregunta de investigación del grupo: ¿cuál de las estructuras resulta más adecuada cuando se pasa de consultar tareas individuales a consultar las relaciones entre una tarea y sus descendientes? Las estructuras comparadas son una lista enlazada simple y un árbol n-ario, y la operación que depende de los descendientes es el cálculo del trabajo contenido en una tarea: su esfuerzo total y su esfuerzo pendiente.

El objetivo general es determinar cuál de las dos estructuras es más adecuada para cada etapa del curso, a partir de fuentes técnicas y de evidencia propia reproducible. Los objetivos específicos son los siguientes:

1. Documentar la práctica de descomposición jerárquica del trabajo y su presencia en herramientas de seguimiento.
2. Implementar en C++11 un prototipo con ambas estructuras, la búsqueda por identificador y el cálculo del trabajo contenido en una tarea.
3. Verificar la corrección de ambas estructuras y comparar los nodos visitados y las comparaciones en cada carga.
4. Recomendar una estructura para cada carga, considerando el costo de transformar los datos si se cambia de estructura.

Antes de ejecutar las pruebas se predijo que (a) en la carga base ambas estructuras harían prácticamente el mismo trabajo, porque un árbol de un solo nivel se recorre como una lista; (b) en la carga modificada la búsqueda por identificador seguiría siendo similar, porque ninguna estructura está ordenada por identificador; y (c) el esfuerzo pendiente sería mucho más costoso en la lista, que debe recorrer la colección completa por cada descendiente, mientras que en el árbol el costo sería proporcional al subárbol consultado, con una diferencia que crecería con el tamaño de la carga. La predicción se registró el 27 de agosto de 2026 en el hilo del grupo en el foro de la Investigación 01, antes de la primera ejecución del prototipo.

El estudio se limita a dos estructuras, a tres operaciones (construcción, búsqueda por identificador y cálculo del trabajo contenido en una tarea) y a cargas de 100 a 1000 tareas. No incluye eliminación ni modificación de tareas, persistencia, interfaz gráfica ni medición del tiempo de ejecución.

Contexto profesional y fundamento técnico

La estructura de desglose del trabajo

La organización de los proyectos del curso corresponde a la Estructura de Desglose del Trabajo (EDT), que el Project Management Institute (PMI, 2006) define como una descomposición jerárquica, orientada a entregables, del trabajo que debe realizar un equipo. Cada elemento tiene un solo elemento superior, y su trabajo equivale a la suma del trabajo de sus elementos inferiores (PMI, 2006; Singh, 2025). Por eso la EDT tiene forma de árbol y la consulta del trabajo contenido en un proyecto exige sumar el de todos sus descendientes.

Las herramientas de seguimiento incorporan esta práctica. En GitHub, una tarea (*issue*) puede dividirse en subtareas con hasta ocho niveles de anidamiento y hasta 100 subtareas por tarea, y el avance de las tareas principales y sus subtareas puede consultarse en las vistas de proyectos (GitHub, s.f.). Las cargas de este estudio alcanzan una profundidad máxima de entre 8 y 12 niveles (Tabla 1), por lo que en los tamaños mayores superan ese límite; esto se discute entre las limitaciones. La fuente documenta la necesidad, no la implementación interna de la herramienta.

Estructuras comparadas

La lista enlazada simple es una estructura lineal en la que cada nodo guarda un dato y un enlace al siguiente; localizar un elemento exige recorrerla desde el inicio (Cormen et al., 2009; Joyanes Aguilar, 2003). En el prototipo cada nodo guarda una tarea y la jerarquía existe solo como el campo `idPadre`, como en una herramienta sencilla que registra una fila por tarea con una columna para la tarea de la que depende.

El árbol n-ario admite cualquier cantidad de hijos por nodo. El prototipo usa la representación primer hijo—siguiente hermano, en la que cada nodo enlaza a su primer hijo y a su

siguiente hermano, lo que permite cualquier cantidad de hijos con un número fijo de enlaces (Cormen et al., 2009; Knuth, 1997; Weiss, 1999); cada nodo enlaza además a su padre. Una raíz ficticia con identificador 0 agrupa las tareas de primer nivel.

Justificación de la selección

La lista se eligió porque representa directamente la carga base: tareas independientes que se agregan conforme se asignan y se consultan una por una, sin orden ni tamaño previo. El árbol se eligió porque su forma coincide con la de una EDT: cada tarea tiene un solo padre y un número variable de hijos, y los descendientes de una tarea quedan reunidos en su subárbol. Ninguna estructura se ordenó por identificador, de modo que la comparación aísla el efecto de representar la jerarquía, que es el cambio introducido por la carga modificada.

Metodología

Prototipo y operaciones

El prototipo, escrito en C++11, representa cada tarea con la estructura `Tarea` (identificador, identificador del padre, nombre, esfuerzo en horas y estado) e implementa ambas estructuras como plantillas (`Lista.h`, `Nodo.h`, `Arbol.h` y `NodoArbol.h`). Los contadores se definen en `Contadores.h`, y el resultado de la consulta sobre descendientes, en la estructura `ResumenTrabajo`. Ambas estructuras reciben los mismos datos en el mismo orden y ofrecen tres operaciones:

- *Construcción.* Ambas estructuras rechazan una tarea cuyo padre no existe. La lista, si la tarea tiene padre, lo busca recorriendo desde el inicio y luego agrega la tarea al final mediante un puntero al último nodo. El árbol busca al padre en preorden desde la raíz y enlaza la tarea como último hijo, recorriendo la cadena de hermanos.

- *Búsqueda por identificador.* La lista recorre desde el inicio; el árbol recorre en preorden desde la raíz.
- *Trabajo contenido en una tarea.* Se localiza la tarea (se devuelve `false` si no existe) y se suman el esfuerzo total, el esfuerzo pendiente (el de las tareas no completadas) y la cantidad de tareas de la propia tarea y de todos sus descendientes. La lista recorre la colección completa buscando los nodos cuyo `idPadre` coincide y repite el proceso por cada hijo encontrado; el árbol recorre el subárbol siguiendo los enlaces, sin comparar identificadores.

Cargas y datos

Los datos se generan con el motor Mersenne Twister `std::mt19937` de C++11 (International Organization for Standardization [ISO], 2011; Matsumoto y Nishimura, 1998).

Para cada tamaño n se crea un generador con la semilla $2026 + n$, que produce, en este orden, las tareas, la jerarquía de la carga modificada y las dos secuencias de consultas. Cada tarea tiene un esfuerzo entero uniforme entre 1 y 8 horas y está completada con probabilidad 0.40. La carga base tiene n tareas independientes. La carga modificada usa las mismas n tareas, con el mismo esfuerzo y estado, pero cada tarea a partir de la segunda es independiente con probabilidad 0.20 y, en otro caso, se asigna como hija de una tarea anterior elegida al azar. Así cada padre existe antes que sus hijos y la única diferencia entre las cargas es la jerarquía. No se fijó un número máximo de niveles. Se usaron $n = 100, 500$ y 1000 : el fenómeno ya es visible con 100 tareas y con 1000 su crecimiento es claro sin convertir el prototipo en una prueba de rendimiento (Tabla 1).

Tabla 1

Composición de las cargas modificadas generadas con la semilla 2026 + n

N	Nivel 1	Nivel 2	Nivel 3	Nivel 4	Nivel 5 o +	Prof maxima	Completadas	Máyor subárbol
100	22	18	17	18	25	8	39(39.0%)	47
500	101	95	87	75	142	10	203(40.6%)	150
1000	189	175	152	135	349	12	390(39.0%)	256

Nota. El nivel 1 corresponde a las tareas independientes y a los proyectos. «Mayor subárbol» es la cantidad de descendientes de la tarea con más descendientes: la tarea 1 con $n = 100$ y 500, y la tarea 2 con $n = 1000$. Las cargas base tienen las mismas tareas y, por tanto, las mismas completadas.

Las tareas, la jerarquía y las consultas utilizadas se guardan en `datos_base_N.csv`, `datos_modificada_N.csv` y `consultas_N.csv`, y los parámetros en `parametros.txt`, porque las distribuciones de `<random>` pueden producir valores distintos con otra biblioteca estándar.

Consultas y casos

En cada carga se ejecuta la misma secuencia de consultas sobre ambas estructuras (Tabla 2), y las búsquedas de la carga base y de la modificada usan la misma secuencia de identificadores. La carga base solo incluye búsquedas, porque sin descendientes el trabajo contenido en una tarea sería solo el de la propia tarea. Cada identificador consultado existe con probabilidad 0.90; en otro caso se elige entre $n + 1$ y $2n$, de modo que la consulta no tiene éxito.

Tabla 2*Consultas ejecutadas por cada tamaño de carga*

Consulta	Carga	N=1000	n=500	n=1000
Búsqueda por identificador	Base y modificada	200	200	200
Sin éxito		25	19	22
Trabajo contenido en una Tarea	Modificada	200	200	200
Sin éxito		20	16	13
Sobre tareas sin subtareas		87	106	89
Descendientes promedio		2.69	2.20	3.05

Nota. El promedio de descendientes se calculó sobre las consultas con éxito, a partir de `datos_modificada_N.csv` y `consultas_N.csv`.

Contadores

Se contaron los nodos visitados, es decir, cada acceso de la operación a un nodo de la estructura, incluidos la raíz ficticia del árbol y el recorrido de la cadena de hermanos al insertar, y las comparaciones de identificador, es decir, las veces que se compara un identificador o un identificador de padre con el valor buscado. En la lista ambos contadores coinciden. En el árbol, la suma del subárbol no compara identificadores, por lo que en la consulta sobre descendientes se usa como métrica principal los nodos visitados. Los contadores se reinician al inicio de cada fase (construcción, búsquedas y consultas sobre descendientes) y acumulan el trabajo de todas sus operaciones; los resultados se informan como total por fase o como promedio por consulta. No se midió tiempo de ejecución.

Comprobación funcional

La corrección se verificó en dos niveles. Primero, el programa ejecuta casos fijos con cinco tareas cuyo resultado se calculó a mano: un proyecto con dos niveles de subtareas, uno de sus entregables con dos subtareas, una tarea sin subtareas y una tarea completada. Se comprueban la inserción de las cinco tareas y el rechazo de una tarea cuyo padre no existe; las

búsquedas de la primera tarea, una intermedia, la última, la rechazada y una inexistente; y el trabajo contenido en el proyecto, en el entregable, en una tarea sin subtareas, en una tarea completada y en una tarea inexistente. Segundo, para las cargas generadas el resultado esperado se calcula sin usar ninguna de las dos estructuras: como cada padre tiene un identificador menor que el de sus hijos, basta recorrer las tareas de la última a la primera sumando cada una a su padre. Cada respuesta se compara con ese valor, y la cantidad de respuestas correctas e incorrectas de cada fase se registra en `resultados.csv`.

Entorno y procedimiento

El prototipo se compiló con g++ 13.3.0 en Ubuntu 24.04, con las opciones `-std=c++11 -Wall -Wextra -pedantic` y sin advertencias. Una sola ejecución de prueba realiza primero los casos fijos y después el estudio: genera las cargas, construye ambas estructuras, ejecuta y verifica las consultas, y escribe en `resultados.csv` los contadores de cada fase. Por último, se repitió el estudio cambiando la semilla base a 1, 777 y 12345 para comprobar que las tendencias no dependen de ella. Las tablas de resultados se calcularon a partir de `resultados.csv`.

La predicción se registró en el foro antes de la primera ejecución del prototipo. Después, el prototipo se revisó para agregar los contadores, la consulta sobre descendientes y la verificación automática, y el estudio se ejecutó de nuevo con la versión descrita aquí; la predicción no se modificó.

Resultados

Corrección

Los 32 casos fijos terminaron sin fallos en ambas estructuras. En el estudio, las 10 000 respuestas de las cargas generadas coincidieron con el resultado esperado: para cada tamaño y

estructura, todas las inserciones, las 200 búsquedas de cada carga y las 200 consultas sobre descendientes de la carga modificada. Las semillas 1, 777 y 12345 tampoco produjeron respuestas incorrectas, de modo que los conteos siguientes corresponden a operaciones correctas.

Búsqueda por identificador

En la carga base el árbol visitó exactamente un nodo más que la lista en cada búsqueda: la raíz ficticia, seguida de las tareas en el mismo orden que en la lista (Tabla 3). En la carga modificada el trabajo siguió siendo del mismo orden, pero ya no idéntico, porque el árbol recorre las tareas agrupadas por proyecto y no por identificador: el árbol visitó entre 3.6 % y 12.7 % más nodos que la lista. Como ambas cargas usan la misma secuencia de búsquedas, la lista realizó exactamente el mismo trabajo en las dos.

Tabla 3

Promedio de nodos visitados por búsqueda (lista / árbol)

Carga	n=100	n=500	n=1000
Base	54.3/55.3	254.1/255.1	534.4/535.4
Modificada	54.3/61.2	254.1/273.2	534.4/553.9

Nota. Promedio de 200 búsquedas, incluidas las que no tienen éxito. En ambas estructuras las comparaciones de identificador fueron iguales a los nodos visitados.

Trabajo contenido en una tarea

La consulta sobre los descendientes separa a las estructuras (Tabla 4). La lista visitó entre 6.3 y 7.6 veces más nodos que el árbol, y la diferencia absoluta creció con n , de 330 a 3740 visitas por consulta. En la lista cada consulta cuesta la búsqueda de la tarea más un recorrido completo de las n tareas por la propia tarea y por cada descendiente, de modo que incluso una tarea sin subtareas cuesta más de n visitas. En el árbol cuesta la búsqueda de la tarea más el

tamaño de su subárbol; como las tareas consultadas tienen en promedio entre 2.2 y 3.1 descendientes (Tabla 2), el costo en el árbol es casi el de una búsqueda.

Tabla 4

Trabajo contenido en una tarea en la carga modificada

Medida	n=100	n=500	n=1000
Nodo visitados por consulta, l lista	386.5	1739.9	4309.0
Nodos visitados por consulta, árbol	56.7	276.5	568.7
Razón lista/ árbol	6.8	6.3	7.6
Comparaciones por consulta, árbol	54.3	274.4	565.8

Nota. Promedio de 200 consultas, incluidas las que no tienen éxito. En la lista las comparaciones fueron iguales a los nodos visitados.

Construcción

La lista no visitó nodos al construir la carga base, porque las tareas independientes no requieren buscar un padre. El árbol visitó $n(n+1)/2$ nodos, porque todas las tareas cuelgan de la raíz y cada inserción recorre la cadena completa de hermanos (Tabla 5). En la carga modificada ambas estructuras deben buscar al padre de cada tarea, y el árbol visitó entre 4.7 % y 28.6 % más nodos que la lista. Con las otras semillas y $n = 1000$, el árbol visitó entre 216 146 y 223 487 nodos al construir la carga modificada y la lista entre 191 971 y 203 823; en la consulta sobre descendientes, la lista visitó en promedio entre 3330 y 5153 nodos por consulta y el árbol entre 554 y 599.

Tabla 5

Nodos visitados al construir cada estructura (lista / árbol)

Carga	n=100	n=500	n=1000
Base	0/5050	0/125 250	0/500 500
Modificada	1860/2392	54 274/56 841	203 867/224 159

Nota. Total de las n inserciones. En la carga base el árbol hizo n comparaciones, una por inserción al revisar la raíz; en la modificada hizo 2111, 51 477 y 205 704.

Discusión

Contraste con la predicción y las fuentes

Las tres partes de la predicción se cumplieron. En la carga base, un árbol de un solo nivel es una lista de hermanos colgada de la raíz y no ofrece ventaja en la búsqueda; la única diferencia es la visita a la raíz ficticia. En la búsqueda de la carga modificada, ninguna estructura está organizada por identificador y ambas recorren la colección; las diferencias se deben al orden de recorrido y disminuyeron en proporción al crecer n .

En la consulta sobre descendientes, la diferencia se explica por la representación. La lista guarda la relación padre–hijo como un dato y debe reconstruirla en cada consulta, recorriendo las n tareas por la tarea y por cada descendiente. El árbol guarda la relación como enlaces y reúne a los descendientes en el subárbol. Esto coincide con la propiedad de la EDT según la cual el trabajo de un elemento es la suma del de sus elementos inferiores (PMI, 2006): el árbol permite calcular esa suma recorriendo solo esos elementos.

La predicción anticipaba que la diferencia crecería con el tamaño de la carga. Eso se cumplió en valor absoluto, pero la razón entre ambas estructuras se mantuvo entre 6.3 y 7.6 sin crecer de forma sostenida. La razón depende de cuántos descendientes tienen las tareas consultadas: con padres elegidos al azar, la mayoría de las tareas tiene pocos o ningún

descendiente, y en el árbol domina el costo de encontrar la tarea, que crece con n igual que el recorrido de la lista. Con proyectos más grandes, como los de una EDT real, la ventaja del árbol sería mayor.

La predicción tampoco consideró la construcción. En la carga modificada el árbol cuesta un poco más que la lista porque recorre los hermanos al enlazar cada tarea: con $n = 1000$, 20 292 visitas más. Ese costo se paga una vez, mientras que las consultas se repiten durante el curso: cada consulta sobre descendientes ahorra en promedio 3740 visitas, de modo que la diferencia se compensa después de seis consultas, aunque cada búsqueda cueste en el árbol unas 20 visitas más.

Recomendaciones

Para la carga base se recomienda la lista enlazada simple: realiza el mismo trabajo de búsqueda con un enlace por nodo en lugar de tres, sin raíz ficticia y sin visitar nodos al agregar tareas, mientras que el árbol visitó 500 500 nodos para construir la misma carga con $n = 1000$. Para la carga modificada se recomienda el árbol n -ario: la búsqueda cuesta casi lo mismo, pero la consulta sobre el trabajo contenido en una tarea, que caracteriza esta etapa, cuesta entre 6.3 y 7.6 veces menos.

Recomendar estructuras distintas implica transformar los datos. Si la migración se hace al asignar el primer proyecto, las tareas existentes son independientes y, con la inserción del prototipo, construir el árbol cuesta lo mismo que en la carga base: $n(n + 1)/2$ visitas, 500 500 con 1000 tareas, que se compensan después de unas 134 consultas sobre descendientes. Si se posterga, el costo de construir el árbol es similar al de la carga modificada, pero mientras tanto cada consulta sobre descendientes se paga con el costo de la lista. Conviene, por tanto, migrar al aparecer el primer proyecto. El costo de la migración proviene de recorrer la cadena de hermanos

al insertar; guardar un enlace al último hijo lo reduciría, pero queda fuera del prototipo estudiado.

Limitaciones y amenazas a la validez

Los contadores miden trabajo lógico, no tiempo ni efectos de memoria caché. En el árbol la suma del subárbol no registra comparaciones, por lo que la consulta sobre descendientes se comparó por nodos visitados. La raíz ficticia añade una visita al árbol en cada búsqueda, lo que favorece levemente a la lista sin cambiar las conclusiones. Los contadores se acumulan por fase, de modo que los resultados son promedios y no permiten separar casos particulares, como el proyecto con más descendientes.

El generador asigna padres al azar entre tareas anteriores, por lo que las más antiguas acumulan más descendientes, la mayoría de las tareas tiene pocos y la profundidad no está limitada: llegó a 12 niveles con $n = 1000$, más que los ocho que admite GitHub (GitHub, s.f.). Una EDT real puede tener proyectos más uniformes y menos niveles. Las búsquedas de ambas cargas usan la misma secuencia de identificadores, lo que facilita compararlas pero no explora otras distribuciones de consultas. La predicción se registró antes de ejecutar la primera versión del prototipo, no la versión revisada descrita aquí. No se estudiaron eliminaciones, cambios de padre ni actualizaciones de estado; solo se usó g++, y las cargas, de hasta 1000 tareas, no representan un sistema de producción.

Conclusiones

Cuando se pasa de consultar tareas individuales a consultar las relaciones entre una tarea y sus descendientes, el árbol n-ario resulta más adecuado que la lista enlazada simple. Mientras las tareas son independientes y solo se buscan por identificador, ambas estructuras realizan el mismo trabajo de búsqueda y la lista es preferible por su simplicidad y porque no tiene costo de

construcción. La diferencia proviene de cómo se representa la jerarquía: es indiferente mientras ninguna consulta depende de ella, pero cuando aparece una consulta sobre los descendientes, guardarla como enlaces evita reconstruirla en cada consulta.

Se recomienda la lista enlazada simple para la carga base y el árbol n-ario para la carga modificada, con la migración al asignar el primer proyecto. Estas recomendaciones son válidas para colecciones de hasta 1000 tareas como las estudiadas, sin orden por identificador y con consultas sobre el trabajo contenido que se repiten durante el curso. Si predominaran las búsquedas por identificador, convendría estudiar además un índice por identificador.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.
- GitHub. (s.f.). *Adding sub-issues*. GitHub Docs. Recuperado el 8 de octubre de 2026, de <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/adding-sub-issues>
- International Organization for Standardization. (2011). *Information technology — Programming languages — C++* (Norma ISO/IEC 14882:2011).
- Joyanes Aguilar, L. (2003). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (3.^a ed.). McGraw-Hill.
- Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3.^a ed.). Addison-Wesley.
- Matsumoto, M. y Nishimura, T. (1998). Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3–30. <https://doi.org/10.1145/272991.272995>
- Project Management Institute. (2006). *Practice standard for work breakdown structures* (2.^a ed.).
- Singh, R. (2025, 24 de noviembre). *Estructura de desglose del trabajo (EDT): Una guía completa*. Institute of Project Management. <https://instituteprojectmanagement.com/es/blog/estructura-de-desglose-del-trabajo-edt-una-guia-completa/>
- Weiss, M. A. (1999). *Data structures & algorithm analysis in C++* (2.^a ed.). Addison-Wesley.